A GPU implementation of XBeach-NH

S. Biondi[a,*]

[a]*Department of Civil and Coastal Engineering, University of Florida, Gainesville, FL, USA*

Abstract

The non-hydrostatic mode of XBeach (XBeach-NH) resolves individual waves in the nearshore, at a cost that limits it to small domains or short events. This work presents xbnh-gpu, a CUDA port of XBeach (revision 6167) that covers every option of this mode and reads and writes XBeach files unchanged. It was verified against Fortran XBeach with 96 test decks, one per option. On one-dimensional grids results agree to round-off; on two-dimensional grids differences are set by the tolerances of the iterative pressure solvers and fall to 10^{-13} with converged solvers. The verification exposed Fortran behaviors that change results; the GPU code reproduces and documents them. On grids of 0.16 to 8.7 million cells, one laptop-class GPU ran 2.0 to 3.3 times faster in double precision and 3.9 to 7.5 times faster in single precision than XBeach with MPI on a 24-core workstation. Existing XBeach models of runup, overtopping and storm impacts can therefore be run at field scale, and in the ensembles that hazard assessments need, on a single workstation. The code and test suite are openly available.

Keywords: XBeach, non-hydrostatic wave model, GPU computing, code verification, nearshore hydrodynamics, coastal hazards

1. Introduction

Phase-resolving models of the nearshore compute individual waves rather than their energy, so they represent infragravity motions, wave groups, runup and overtopping without the parameterizations that phase-averaged models need. Non-hydrostatic shallow-water models add a non-hydrostatic pressure to the nonlinear shallow-water equations and resolve frequency dispersion with one or a few vertical layers (Stelling and Zijlema, 2003; Zijlema and Stelling, 2008; Zijlema et al., 2011). The non-hydrostatic mode of XBeach (Roelvink et al., 2009; Smit et al., 2013) is of this type. It is widely used for runup and storm impacts on sandy and gravel coasts, reefs and dikes (McCall et al., 2014; Roelvink et al., 2018; Lashley et al., 2018, 2020; de Beer et al., 2021), and increasingly for morphodynamics (Elsayed et al., 2022), and its reduced two-layer formulation extends accurate dispersion to deeper water at little extra cost (de Ridder et al., 2021).

Cost limits where these models can be used. Each wave must be resolved by several grid cells and the explicit time step falls with the grid spacing, so a storm simulation over a few kilometers of coast at 2 m resolution needs millions of cells and hundreds of thousands of time steps. In engineering practice the cost decides whether a phase-resolving model can be used at all: design and assessment studies of dikes, dunes and barriers need many storm conditions, sensitivity runs and, increasingly, probabilistic ensembles, and in benchmarks of overtopping at dikes the computational cost of phase-resolving models weighs against their accuracy (Lashley et al., 2020). On reef-lined coasts the non-hydrostatic mode of XBeach predicts runup better than the wave-averaged mode but at a much higher cost (Quataert et al., 2020), and hazard assessments built on it need thousands of runs (Pearson et al., 2017; Scott et al., 2020). XBeach parallelizes by MPI domain decomposition; on one workstation its speed is bounded by the memory bandwidth that the cores share, and on a cluster by the communication between subdomains.

Graphics processing units (GPUs) offer several times the memory bandwidth of a CPU socket, and stencil-based coastal solvers have been ported to them with large gains, for shallow-water flow (Brodtkorb et al., 2012; Qin et al.,

*Corresponding author.
Email address: stefano.biondi@ufl.edu (S. Biondi)

2019; Morales-Hernández et al., 2021), Boussinesq-type waves (Tavakkol and Lynett, 2017; Kim et al., 2018; Yuan et al., 2020) and non-hydrostatic shallow flow (Escalante et al., 2018). For XBeach, a GPU implementation exists for the wave-averaged (surf beat) mode (Rautenbach et al., 2022) but not for the non-hydrostatic mode. Most GPU coastal models are new codes that share equations, but not results, with an established CPU model. For a model with a large user base this matters: XBeach decks encode calibrated choices of friction, breaking and boundary forcing, and a code that solves similar equations differently cannot reuse them without recalibration. A port is useful to existing users only if it reproduces the Fortran model closely enough that the choice between the two is a question of speed.

For atmospheric and ocean models, a port is usually accepted when its departure from the original stays within the growth of round-off perturbations (Rosinski and Williamson, 1997) or when it passes a statistical test against an ensemble of runs of the original (Baker et al., 2015), because the chaotic growth of round-off makes identical results impossible after days of simulated time. A nearshore model run for minutes to hours allows a stricter test. Where the numerical method has no iterative component, the port can be required to match the original to round-off for every option; where it has one, the difference can be required to shrink with the iteration tolerance.

This paper describes xbnh-gpu, a CUDA implementation of XBeach (trunk revision 6167) in non-hydrostatic mode. It covers every option that XBeach accepts with `wavemodel = nonh` and reads and writes XBeach files unchanged. Section 2 summarizes the model and the port, Section 3 verifies the port against the Fortran code option by option and with a solver-tolerance convergence test, Section 4 reports the computational cost, and Section 5 discusses the limits of the agreement and of the code and what the port means for coastal engineering practice.

2. Model and implementation

2.1. XBeach in non-hydrostatic mode

In non-hydrostatic mode XBeach solves the depth-averaged continuity and momentum equations with a non-hydrostatic pressure, following the one-layer Keller-box approach of Stelling and Zijlema (2003) as described by Smit et al. (2013). With water depth $h = \zeta + d$, surface elevation ζ, still-water depth d, depth-averaged velocity (u, v) and non-hydrostatic pressure q_b at the bed (zero at the surface), the cross-shore momentum equation and the vertical momentum and local continuity equations read

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = -g\frac{\partial \zeta}{\partial x} - \frac{1}{h}\left(\frac{\partial (h\bar{q})}{\partial x} - q_b\frac{\partial d}{\partial x}\right) - \frac{\tau_{bx}}{\rho h} + \nu_h\nabla^2 u + F_x \tag{1}$$

$$\frac{\partial w_s}{\partial t} + \frac{\partial w_b}{\partial t} = \frac{2q_b}{h}, \qquad \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} + \frac{w_s - w_b}{h} = 0 \tag{2}$$

where $\bar{q} = q_b/2$ is the depth-averaged pressure, w_s and w_b are the vertical velocities at the surface and the bed ($w_b = -u\,\partial d/\partial x - v\,\partial d/\partial y$), τ_{bx} is the bed shear stress, ν_h the horizontal viscosity and F_x the sum of wind, vegetation, ship and other forces; the alongshore equation is analogous. Each time step computes a hydrostatic predictor, solves a Poisson equation for q_b that enforces Equation (2) and corrects the velocities. Breaking follows the hydrostatic front approximation of Smit et al. (2013): the pressure is switched off where the surface rises faster than a fraction of $\sqrt{gh}$, and breaking cells receive extra viscosity. The reduced two-layer option (de Ridder et al., 2021) splits the water column into two layers with one pressure unknown at the interface, which extends accurate dispersion to $kh \approx 4$.

The other processes act through source terms and the bed. Spectral wave boundaries are synthesized as sums of wave components, optionally with bound long waves. Vegetation adds a drag force per plant section (van Rooijen et al., 2016), and a porous canopy model resolves the flow inside the canopy. Groundwater exchanges water with the surface by infiltration and exfiltration (McCall et al., 2014; Almeida et al., 2017), and ships add a moving pressure field (Almström et al., 2021). Sediment transport solves an advection-diffusion equation for the suspended concentration of each fraction, with equilibrium concentrations from the chosen transport formula; the bed follows the transport divergence, with avalanching on steep slopes and an optional non-erodible layer. Gravel formulations and the XBeach-G settings (McCall et al., 2014, 2015) change friction, breaking and transport on gravel beaches.

2.2. Port design

xbnh-gpu translates XBeach routine by routine. Each routine becomes one or more CUDA kernels in which one thread updates one grid point, launched in the order in which XBeach calls the routines in its time loop. Arrays keep the XBeach memory layout, with the cross-shore index varying fastest, so input and output files map to device memory without reordering. Scalars and device pointers sit in constant memory, and grid metrics are stored as arrays only for non-uniform grids. On a uniform grid the terms that depend on the difference in orientation between neighboring grid lines are skipped; they are exactly zero there, so the shortcut changes no bit of the result. The program is one translation unit of about 8,300 lines, with one header per XBeach module, compiled for double or single precision; in single precision the time, the grid coordinates and the boundary series stay in double precision.

Three parts of XBeach are sequential by construction and were replaced.

1. The non-hydrostatic pressure equation is a five-diagonal system that XBeach solves with the strongly implicit procedure (SIP) of Stone (1968), an incomplete factorization swept in grid order. xbnh-gpu solves the same system with BiCGStab (van der Vorst, 1992) and a Jacobi preconditioner, to a relative residual of 10^{-3} by default, because incomplete-factorization preconditioners lose most of their benefit on GPUs while a Jacobi preconditioner is fully parallel (Li and Saad, 2012; Anzt et al., 2017). Both solvers are iterative, so in 2D the codes agree only to the solver tolerances (Section 3.2). On 1D grids the system is tridiagonal: XBeach solves it directly, and xbnh-gpu uses parallel cyclic reduction (Zhang et al., 2010), which is also direct. The groundwater head is treated in the same way.
2. Avalanching updates the bed in place while it sweeps the grid, so the result depends on the sweep order. xbnh-gpu assigns one thread to each grid row for the cross-shore sweep and one to each column for the alongshore sweep, which preserves the order along each line.
3. The spectral boundary synthesis draws phases and directions on the CPU with the XBeach random number generator, so that `random = 0` gives the XBeach series, and computes the sums over wave components for every boundary point and time on the GPU. These sums dominate the cost of the synthesis, which takes hours on one CPU core for long boundaries.

Agreement to round-off needs more than the same equations. The port reproduces the arithmetic of the Fortran code where it affects the result: single-precision constants in comparisons, the evaluation order of integer powers, loop variables that keep their last value and are used after the loop, and the order in which neighboring cells are read and written within a routine. Time stepping follows XBeach exactly, including the shortening of the time step to land on output times, because a different sequence of time steps changes the solution at round-off level and the difference then grows.

2.3. XBeach behaviors reproduced

The verification exposed behaviors of XBeach r6167 that change results and that a reimplementation from the equations would not reproduce. xbnh-gpu reproduces them by default, so that existing decks give the same answers, and documents each one (Table 1). Where the behavior is a crash or depends on undefined memory, xbnh-gpu stops with a message or runs with a warning instead.

Table 1: XBeach r6167 behaviors in non-hydrostatic mode and their treatment in xbnh-gpu.

Area	XBeach behavior	xbnh-gpu
Time-series boundary	The first call of the boundary routine opens the file and then advances it, so the first step interpolates between records	reproduced
Cell-center velocity	v_e lags one flow step, because it is computed before the face velocity is updated; it feeds output and the canopy model	reproduced
1D wind	In 1D runs a cross-shore wind also enters the alongshore stress component, so the stress is $\sqrt{2}$ too large	reproduced
Transport formulas	`mpm`, `wilcock_crow`, `engelund_fredsoe`, `wong_parker`, `fl_vb` and `fredsoe_deigaard` are accepted but never called, so they give no transport	reproduced
Bed load	The bed-load fluxes of the sand formulas are zero in non-hydrostatic mode, so sand moves as suspended load only	reproduced
`intrasedtr`	The suspended-load term is never set, so the formula gives bed load only	reproduced
`nielsen2006`	The run fails unless `streaming = 1`	runs, with a warning
Hotstart	Velocities read from the hotstart files are set to zero during initialization	reproduced with `gpu_hotstart_keepuv = 0`; kept by default
Placeholder defaults	Parameters that a deck does not set can keep the value −123 and enter the computation, for example `tintm` in the weight of the first averaging interval	reproduced
2D avalanching	With a time-series boundary `Hrms` is not read and keeps the value −123, so the swash excursion that scales the alongshore avalanching flux is the square root of a negative number; wherever an alongshore slope exceeds the critical slope, the whole erodible layer moves in one step and the run fails within a few steps	reproduced
`highcomp`	Second-order components take their phases from the frequency bin above each component, a row of the transfer matrix is cleared when $k_3 h > 5$ at its first entry, and no alongshore flux is added	reproduced
`gwfastsolve`	The matrix is filled with an uninitialized index	stops with a message
Cyclic laterals	Under MPI the velocities are not exchanged after the pressure correction, so the continuity equation at subdomain seams uses uncorrected halo velocities	not reproduced: one domain, exchange after every update

3. Verification

3.1. Test decks

xbnh-gpu was compared with the serial Fortran build of XBeach r6167 on identical input. A generator script builds one deck per option, or group of options, from a common base case: a barred beach 520 m long with a 1:50 slope, a shore-parallel bar 300 m from the offshore boundary, a grid spacing of 2 m and 120 s of irregular waves (significant wave height 1 m, peak period 8 s) entering through a time-series boundary. Two-dimensional (2D) decks have 61 alongshore rows and a bar that is lowered toward one lateral boundary; one-dimensional (1D) decks are single cross-shore transects. The 96 decks (43 in 1D, 53 in 2D) cover the grid types, the one- and two-layer flow models, every lateral, offshore and back boundary type, the water-level options, every friction law, wind, discharges, rainfall, every wave boundary type including the four spectral inputs and second-order components, every output type, vegetation, groundwater, ships, the sediment transport formulas, multiple fractions, a non-erodible layer, avalanching, the morphological acceleration factor, prescribed bed changes and the XBeach-G options. Where an option left the XBeach result unchanged, the deck was modified until it did not, so that every deck exercises its option.

Differences are measured on the global output over $t \geq 40$ s, in cells that stay wet away from the grid boundaries: the root-mean-square (RMS) difference of the significant wave height $H_s = 4\sigma(\zeta)$ relative to the mean H_s; the RMS difference of the mean current relative to the RMS mean current; the RMS difference of the instantaneous water level; the RMS difference of the final bed relative to the RMS bed change; and the RMS difference of the suspended concentration relative to its RMS value. Every other output file (point and runup-gauge records, mean, variance, minimum and maximum fields, drifter tracks and ship forces) is compared by its largest difference relative to its largest value.

3.2. Results

All 96 decks pass (Table 2, Figure 1). The 1D decks agree with XBeach to round-off: the relative differences in wave height, current and concentration are below 10^{-14}, and the final bed differs by at most 5.5×10^{-13} of the

Table 2: Largest and median differences between xbnh-gpu (double precision) and XBeach over the 96 test decks.

Metric	1D largest	1D median	2D largest	2D median
H_s, relative RMS	5.5×10^{-15}	1.5×10^{-15}	4.9×10^{-4}	1.4×10^{-5}
Mean current, relative RMS	4.2×10^{-15}	9.8×10^{-16}	7.2×10^{-4}	4.4×10^{-5}
Water level, RMS (m)	3.4×10^{-15}	1.4×10^{-15}	3.7×10^{-4}	2.4×10^{-5}
Bed level, relative RMS	5.5×10^{-13}	9.2×10^{-14}	2.8×10^{-4}	8.1×10^{-5}
Concentration, relative RMS	9.0×10^{-15}	7.4×10^{-15}	9.4×10^{-5}	5.8×10^{-5}

bed change, including the decks with several sediment fractions, gravel formulations, groundwater and the XBeach-G settings. The 2D decks agree to 3×10^{-6} to 7×10^{-4} in wave height and current, with medians of 1 to 4×10^{-5}, and to 3×10^{-4} or better in bed change. For scale, XBeach itself run with four MPI ranks differs from serial XBeach by 3.6×10^{-3} in wave height and 1.6×10^{-2} in mean current on a 39,000-cell rip-channel deck of the same type.

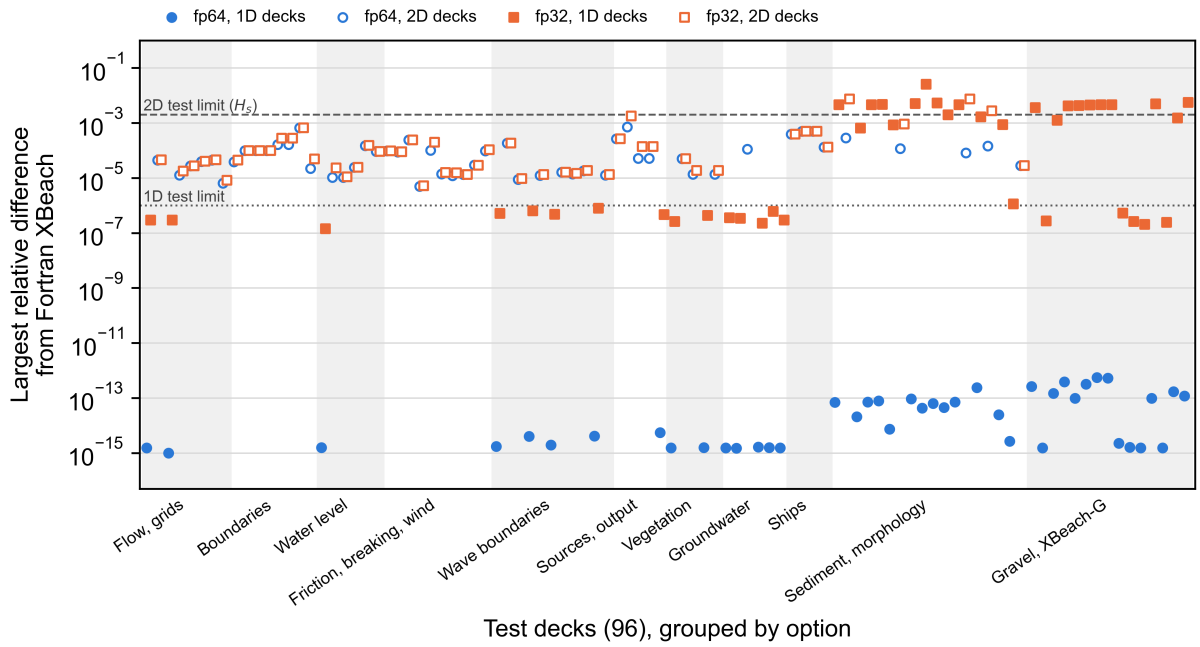


Figure 1: Largest relative difference from Fortran XBeach for each test deck, grouped by option, in double (circles) and single precision (squares). Filled markers are 1D decks and open markers 2D decks; values at round-off are drawn at 10^{-16}.

The 2D differences come from the pressure solvers, not from the implementation. The 2D sediment deck was run with both solver tolerances set to the same value, from 10^{-3} to 10^{-11} (Figure 2). The largest difference over the run in water level, velocity, concentration and bed level falls in proportion to the tolerance and reaches 3×10^{-13} or less at 10^{-11}. At the default tolerances of the two codes (5×10^{-3} with at most 30 iterations in XBeach, 10^{-3} in xbnh-gpu) the largest relative differences are 5×10^{-3} in water level, 2×10^{-3} in velocity, 3×10^{-4} in concentration and 2×10^{-6} in bed level. At equal nominal tolerance the two solvers do not reach equal accuracy on every step: on the first step of a moving ship, a sharp local forcing, BiCGStab needs a residual two orders of magnitude smaller than SIP, and with the GPU tolerance at 10^{-13} the ship deck with waves also agrees to 10^{-14}.

The boundary series that xbnh-gpu synthesizes from spectra with `random = 0` are those of XBeach: the 1D files are identical and the 2D files agree to the last printed digit. The Fortran output files are identical in name, size and layout, the netCDF files have the same dimensions, variables and attributes, and hotstart files written by either code restart the other to round-off.

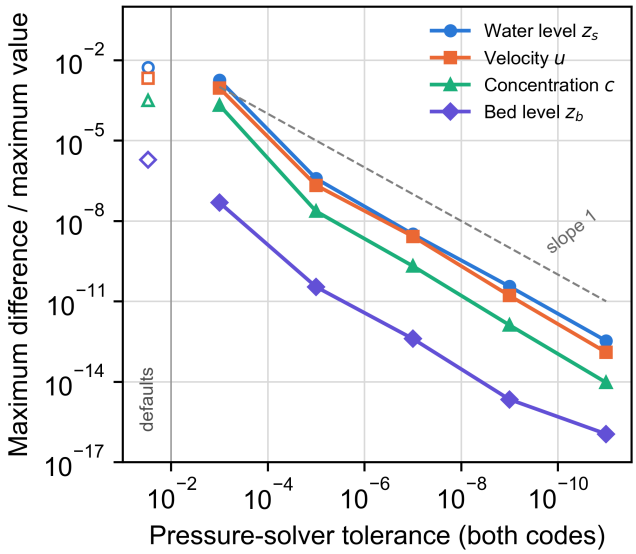


Figure 2: Largest difference between xbnh-gpu and XBeach over a 120 s run of the 2D sediment deck, relative to the largest value of each field, against the tolerance of both pressure solvers. Open markers: default tolerances of the two codes. The dashed line has slope one.

3.3. Single precision

In single precision the 1D decks agree with XBeach to 2×10^{-6} in wave height, current and water level, and the 2D decks as well as in double precision, because there the solver tolerance dominates. Bed changes differ more, by 0.5 percent of the change after 120 s in the median deck and by up to 2.5 percent: the bed integrates small transport increments over many steps, the case in which reduced precision is known to fail in other geophysical models (Dawson et al., 2017). Single precision therefore suits hydrodynamic runs, as it suits weather forecasting (Váňa et al., 2017), and double precision is the safer choice for morphological runs.

4. Computational performance

Timings were measured on one workstation with an Intel Core i9-14900K processor (8 performance and 16 efficiency cores, 32 threads) and an NVIDIA RTX 2000 Ada laptop-class GPU with 16 GB of memory, under Windows Subsystem for Linux. XBeach was built with gfortran and Open MPI and run with 2 to 32 ranks. For both codes the time is the duration of the time loop, which XBeach reports itself.

The benchmark deck is a rip-channel beach stretched alongshore from 0.16 to 8.7 million cells and run for 30 s (Figure 3). XBeach stops gaining speed beyond about 8 ranks, because all cores share the memory bandwidth of one socket, as expected for stencil codes of low arithmetic intensity (Williams et al., 2009); its best cost per cell and time step grows from 8.8×10^{-8} s at 0.16 million cells to 1.4×10^{-7} s at 8.7 million cells. The GPU cost is nearly independent of grid size, 4.2 to 4.5×10^{-8} s in double precision and 1.9 to 2.2×10^{-8} s in single precision, which is 2.0 to 3.3 times (double) and 3.9 to 7.5 times (single) less than the best XBeach run, with the larger factors on the larger grids. The pressure solve takes half of the GPU time in double precision and two thirds in single precision.

The benchmark deck uses the hydrodynamic options only; the cost of the other processes on large grids was not measured. Data-center GPUs execute double precision at a much higher rate than this card and have several times its memory bandwidth, so larger speed-ups are expected there; they were not measured.

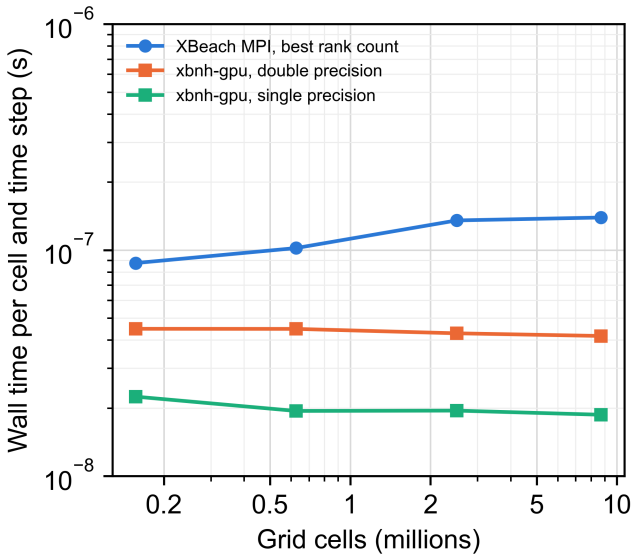


Figure 3: Wall time per cell and time step against grid size for XBeach with MPI (best rank count on the workstation) and for xbnh-gpu in double and single precision.

5. Discussion

5.1. What the agreement means

The verification separates two sources of difference. On 1D grids both codes solve the pressure equation directly, and the GPU code reproduces XBeach to round-off over the whole run for every option. A translation error in any routine would appear there far above round-off, because the 1D decks exercise the same kernels as the 2D decks except for the lateral boundaries and the alongshore terms. On 2D grids both codes stop their pressure iterations at a tolerance, and the differences are those of two truncated solutions; they fall with the tolerance to round-off (Section 3.2), so they do not come from the implementation. At the default tolerances they are at least an order of magnitude smaller than the difference that XBeach shows between serial and MPI runs of the same deck.

Bitwise identity is attainable in neither code. Two runs of xbnh-gpu on the same GPU differ by about 10^{-14}, because the dot products of the solver are summed with atomic operations whose order changes from run to run (Collange et al., 2015), and XBeach under MPI changes its results with the number of ranks. In 2D, wave breaking amplifies small differences, so agreement wave by wave at a point decays over long runs, as it does between XBeach runs with different rank counts; the statistics that users take from these models (wave heights, mean currents, setup and bed change) are what the comparison should be judged on.

5.2. Reproducing XBeach behaviors

xbnh-gpu reproduces XBeach behaviors that a user may regard as errors (Table 1), because an existing deck must give the answer it gives in XBeach, including the effect of any calibration that compensated for such behaviors. The list is useful on its own. Six of the transport formulas that XBeach r6167 accepts never move sediment, the bed load of the sand formulas is discarded in non-hydrostatic mode, the wind stress in 1D runs is too large by a factor of $\sqrt{2}$, velocities read from hotstart files are discarded, and 2D avalanching fails with time-series wave boundaries. A user unaware of these behaviors may attribute their effects to physics. Behaviors that crash or depend on uninitialized memory are not reproduced.

5.3. Implications for coastal engineering practice

Because xbnh-gpu runs XBeach decks unchanged and reproduces their results, a model that has been set up and calibrated for a site, for example for runup on a reef-lined coast, overtopping of a dike with a shallow foreshore or the storm response of a gravel barrier (Quataert et al., 2020; Lashley et al., 2020; Almeida et al., 2017), can be moved to the GPU without recalibration; the test suite provides the check that the option set of the deck is reproduced. The speed-up applies to the runs that dominate the cost of engineering studies. A set of design storms, the sensitivity runs that accompany them, and the ensembles of a probabilistic hazard assessment (Pearson et al., 2017; Scott et al., 2020) can be run on one workstation with a single GPU rather than on a cluster, and a 16 GB card holds about 20 million cells in double precision with the hydrodynamic options (Section 5.4), or about 80 km^2 of coast at 2 m resolution, in one domain.

Two choices follow from the verification. Single precision reproduces wave heights and mean currents closely enough for hydrodynamic studies and roughly doubles the speed, while double precision should be kept for morphological runs, in which bed changes in single precision differ by up to a few percent. Users should also check their decks against the XBeach behaviors of Table 1: a deck that selects a transport formula XBeach never calls, or a 1D run with wind, gives the same answer in both codes, but not the answer the user intended.

5.4. Limitations

xbnh-gpu runs on one GPU, and memory sets the largest grid. With the hydrodynamic options the code needs about 0.8 kB per cell in double precision and 0.43 kB in single precision (6.7 and 3.6 GB for 8.7 million cells), so a 16 GB card holds about 20 and 37 million cells; larger domains need several GPUs, which the code does not support. The card used here executes double-precision arithmetic at 1/64 of its single-precision rate, yet single precision is only 2.2 times faster, because the cost is set by memory traffic, which halves, rather than by arithmetic. Bed changes in single precision differ by up to a few percent (Section 3.3), so double precision remains the choice for morphological runs.

The port covers the non-hydrostatic mode only. The wave-action modes of XBeach (surf beat and stationary) share the flow, sediment and morphology routines but add a wave-action solver on directional bins, which is not part of this code. The verification is against one revision of XBeach; later revisions change some routines, and the test suite with its stored XBeach statistics is the tool for checking the GPU code against a new revision.

6. Conclusions

xbnh-gpu is a CUDA implementation of XBeach revision 6167 in non-hydrostatic mode. It covers every option XBeach accepts in that mode and runs XBeach decks unchanged on one GPU. The port follows the Fortran code routine by routine and replaces only the three parts that are sequential by construction: the incomplete-factorization pressure solver, the in-place avalanching sweeps and the spectral boundary synthesis.

On 96 test decks, one per option, the GPU code reproduces XBeach to round-off on 1D grids, where both codes solve the pressure equation directly, for every option including sediment transport, morphology, groundwater and the XBeach-G settings. On 2D grids the differences are set by the tolerances of the two iterative pressure solvers and fall with them to round-off; at the default tolerances they are about 10^{-5} in wave height and mean current, far below the differences between serial and MPI runs of XBeach.

The verification documented XBeach behaviors that change results, among them transport formulas that are accepted but never called, a wind stress that is too large in 1D runs and an alongshore avalanching flux that is undefined with time-series wave boundaries. The GPU code reproduces them by default, so that calibrated decks keep their results, and lists them so that users can check whether their decks depend on them.

On one laptop-class GPU the code ran 2.0 to 3.3 times faster in double precision and 3.9 to 7.5 times faster in single precision than XBeach with MPI on a 24-core workstation, with the larger gains on the larger grids. Single precision suits hydrodynamic runs, and double precision is recommended for morphological runs. The test suite and its stored XBeach statistics allow the agreement to be checked on any GPU and against later XBeach revisions.

7. Code availability

xbnh-gpu version 2.0 is distributed under the GNU Lesser General Public License 3.0, the license of XBeach, at https://github.com/stebiondi/xbnh-gpu. The repository contains the source code, the build and installation scripts, the script that fetches and builds the reference Fortran XBeach r6167, the test decks and their stored XBeach statistics, and a user guide. The code requires an NVIDIA GPU of compute capability 6.0 or newer and CUDA 12.

CRediT authorship contribution statement

S. Biondi: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Almeida, L.P., Masselink, G., McCall, R., Russell, P., 2017. Storm overwash of a gravel barrier: Field measurements and XBeach-G modelling. Coastal Engineering 120, 22–35. doi:10.1016/j.coastaleng.2016.11.009.

Almström, B., Roelvink, D., Larson, M., 2021. Predicting ship waves in sheltered waterways – An application of XBeach to the Stockholm Archipelago, Sweden. Coastal Engineering 170, 104026. doi:10.1016/j.coastaleng.2021.104026.

Anzt, H., Gates, M., Dongarra, J., Kreutzer, M., Wellein, G., Köhler, M., 2017. Preconditioned Krylov solvers on GPUs. Parallel Computing 68, 32–44. doi:10.1016/j.parco.2017.05.006.

Baker, A.H., Hammerling, D.M., Levy, M.N., Xu, H., Dennis, J.M., Eaton, B.E., Edwards, J., Hannay, C., Mickelson, S.A., Neale, R.B., Nychka, D., Shollenberger, J., Tribbia, J., Vertenstein, M., Williamson, D., 2015. A new ensemble-based consistency test for the Community Earth System Model (pyCECT v1.0). Geoscientific Model Development 8, 2829–2840. doi:10.5194/gmd-8-2829-2015.

de Beer, A., McCall, R., Long, J., Tissier, M., Reniers, A., 2021. Simulating wave runup on an intermediate–reflective beach using a wave-resolving and a wave-averaged version of XBeach. Coastal Engineering 163, 103788. doi:10.1016/j.coastaleng.2020.103788.

Brodtkorb, A.R., Sætra, M.L., Altinakar, M., 2012. Efficient shallow water simulations on GPUs: Implementation, visualization, verification, and validation. Computers & Fluids 55, 1–12. doi:10.1016/j.compfluid.2011.10.012.

Collange, C., Defour, D., Graillat, S., Iakymchuk, R., 2015. Numerical reproducibility for the parallel reduction on multi- and many-core architectures. Parallel Computing 49, 83–97. doi:10.1016/j.parco.2015.09.001.

Dawson, A., Düben, P.D., MacLeod, D.A., Palmer, T.N., 2017. Reliable low precision simulations in land surface models. Climate Dynamics 51, 2657–2666. doi:10.1007/s00382-017-4034-x.

Elsayed, S.M., Gijsman, R., Schlurmann, T., Goseberg, N., 2022. Nonhydrostatic Numerical Modeling of Fixed and Mobile Barred Beaches: Limitations of Depth-Averaged Wave Resolving Models around Sandbars. Journal of Waterway, Port, Coastal, and Ocean Engineering 148. doi:10.1061/(asce)ww.1943-5460.0000685.

Escalante, C., Morales de Luna, T., Castro, M., 2018. Non-hydrostatic pressure shallow flows: GPU implementation using finite volume and finite difference scheme. Applied Mathematics and Computation 338, 631–659. doi:10.1016/j.amc.2018.06.035.

Kim, B., Oh, C., Yi, Y., Kim, D.H., 2018. GPU-Accelerated Boussinesq Model Using Compute Unified Device Architecture FORTRAN. Journal of Coastal Research 85, 1176–1180. doi:10.2112/si85-236.1.

Lashley, C.H., Roelvink, D., van Dongeren, A., Buckley, M.L., Lowe, R.J., 2018. Nonhydrostatic and surfbeat model predictions of extreme wave run-up in fringing reef environments. Coastal Engineering 137, 11–27. doi:10.1016/j.coastaleng.2018.03.007.

Lashley, C.H., Zanuttigh, B., Bricker, J.D., van der Meer, J., Altomare, C., Suzuki, T., Roeber, V., Oosterlo, P., 2020. Benchmarking of numerical models for wave overtopping at dikes with shallow mildly sloping foreshores: Accuracy versus speed. Environmental Modelling & Software 130, 104740. doi:10.1016/j.envsoft.2020.104740.

Li, R., Saad, Y., 2012. GPU-accelerated preconditioned iterative linear solvers. The Journal of Supercomputing 63, 443–466. doi:10.1007/s11227-012-0825-3.

McCall, R., Masselink, G., Poate, T., Roelvink, J., Almeida, L., 2015. Modelling the morphodynamics of gravel beaches during storms with XBeach-G. Coastal Engineering 103, 52–66. doi:10.1016/j.coastaleng.2015.06.002.

McCall, R., Masselink, G., Poate, T., Roelvink, J., Almeida, L., Davidson, M., Russell, P., 2014. Modelling storm hydrodynamics on gravel beaches with XBeach-G. Coastal Engineering 91, 231–250. doi:10.1016/j.coastaleng.2014.06.007.

Morales-Hernández, M., Sharif, M.B., Kalyanapu, A., Ghafoor, S., Dullo, T., Gangrade, S., Kao, S.C., Norman, M., Evans, K., 2021. TRITON: A Multi-GPU open source 2D hydrodynamic flood model. Environmental Modelling & Software 141, 105034. doi:10.1016/j.envsoft.2021.105034.

Pearson, S.G., Storlazzi, C.D., van Dongeren, A.R., Tissier, M.F.S., Reniers, A.J.H.M., 2017. A Bayesian-Based System to Assess Wave-Driven Flooding Hazards on Coral Reef-Lined Coasts. Journal of Geophysical Research: Oceans 122, 10099–10117. doi:10.1002/2017jc013204.

Qin, X., LeVeque, R.J., Motley, M.R., 2019. Accelerating an Adaptive Mesh Refinement Code for Depth-Averaged Flows Using GPUs. Journal of Advances in Modeling Earth Systems 11, 2606–2628. doi:10.1029/2019ms001635.

Quataert, E., Storlazzi, C., van Dongeren, A., McCall, R., 2020. The importance of explicitly modelling sea-swell waves for runup on reef-lined coasts. Coastal Engineering 160, 103704. doi:10.1016/j.coastaleng.2020.103704.

Rautenbach, C., Trenham, C., Benn, D., Hoeke, R., Bosserelle, C., 2022. Computing efficiency of XBeach hydro- and wave dynamics on Graphics Processing Units (GPUs). Environmental Modelling & Software 157, 105532. doi:10.1016/j.envsoft.2022.105532.

de Ridder, M.P., Smit, P.B., van Dongeren, A.R., McCall, R.T., Nederhoff, K., Reniers, A.J., 2021. Efficient two-layer non-hydrostatic wave model with accurate dispersive behaviour. Coastal Engineering 164, 103808. doi:10.1016/j.coastaleng.2020.103808.

Roelvink, D., McCall, R., Mehvar, S., Nederhoff, K., Dastgheib, A., 2018. Improving predictions of swash dynamics in XBeach: The role of groupiness and incident-band runup. Coastal Engineering 134, 103–123. doi:10.1016/j.coastaleng.2017.07.004.

Roelvink, D., Reniers, A., van Dongeren, A., van Thiel de Vries, J., McCall, R., Lescinski, J., 2009. Modelling storm impacts on beaches, dunes and barrier islands. Coastal Engineering 56, 1133–1152. doi:10.1016/j.coastaleng.2009.08.006.

van Rooijen, A.A., McCall, R.T., van Thiel de Vries, J.S.M., van Dongeren, A.R., Reniers, A.J.H.M., Roelvink, J.A., 2016. Modeling the effect of wave-vegetation interaction on wave setup. Journal of Geophysical Research: Oceans 121, 4341–4359. doi:10.1002/2015jc011392.

Rosinski, J.M., Williamson, D.L., 1997. The Accumulation of Rounding Errors and Port Validation for Global Atmospheric Models. SIAM Journal on Scientific Computing 18, 552–564. doi:10.1137/s1064827594275534.

Scott, F., Antolinez, J.A.A., McCall, R., Storlazzi, C., Reniers, A., Pearson, S., 2020. Hydro-Morphological Characterization of Coral Reefs for Wave Runup Prediction. Frontiers in Marine Science 7. doi:10.3389/fmars.2020.00361.

Smit, P., Zijlema, M., Stelling, G., 2013. Depth-induced wave breaking in a non-hydrostatic, near-shore wave model. Coastal Engineering 76, 1–16. doi:10.1016/j.coastaleng.2013.01.008.

Stelling, G., Zijlema, M., 2003. An accurate and efficient finite-difference algorithm for non-hydrostatic free-surface flow with application to wave propagation. International Journal for Numerical Methods in Fluids 43, 1–23. doi:10.1002/fld.595.

Stone, H.L., 1968. Iterative Solution of Implicit Approximations of Multidimensional Partial Differential Equations. SIAM Journal on Numerical Analysis 5, 530–558. doi:10.1137/0705044.

Tavakkol, S., Lynett, P., 2017. Celeris: A GPU-accelerated open source software with a Boussinesq-type wave solver for real-time interactive simulation and visualization. Computer Physics Communications 217, 117–127. doi:10.1016/j.cpc.2017.03.002.

Váňa, F., Düben, P., Lang, S., Palmer, T., Leutbecher, M., Salmond, D., Carver, G., 2017. Single Precision in Weather Forecasting Models: An Evaluation with the IFS. Monthly Weather Review 145, 495–502. doi:10.1175/mwr-d-16-0228.1.

van der Vorst, H.A., 1992. Bi-CGSTAB: A Fast and Smoothly Converging Variant of Bi-CG for the Solution of Nonsymmetric Linear Systems. SIAM Journal on Scientific and Statistical Computing 13, 631–644. doi:10.1137/0913035.

Williams, S., Waterman, A., Patterson, D., 2009. Roofline: an insightful visual performance model for multicore architectures. Communications of the ACM 52, 65–76. doi:10.1145/1498765.1498785.

Yuan, Y., Shi, F., Kirby, J.T., Yu, F., 2020. FUNWAVE-GPU: Multiple-GPU Acceleration of a Boussinesq-Type Wave Model. Journal of Advances in Modeling Earth Systems 12. doi:10.1029/2019ms001957.

Zhang, Y., Cohen, J., Owens, J.D., 2010. Fast tridiagonal solvers on the GPU, in: Proceedings of the 15th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, ACM. pp. 127–136. doi:10.1145/1693453.1693472.

Zijlema, M., Stelling, G., 2008. Efficient computation of surf zone waves using the nonlinear shallow water equations with non-hydrostatic pressure. Coastal Engineering 55, 780–790. doi:10.1016/j.coastaleng.2008.02.020.

Zijlema, M., Stelling, G., Smit, P., 2011. SWASH: An operational public domain code for simulating wave fields and rapidly varied flows in coastal waters. Coastal Engineering 58, 992–1012. doi:10.1016/j.coastaleng.2011.05.015.